

Introducción

En esta práctica analizaremos cómo es posible vulnerar una página web sencilla que contiene una página de inicio de sesión en PHP y que presenta errores que la hacen vulnerable a inyección SQL. A través de esta vulnerabilidad, realizaremos diferentes ataques para comprobar cómo un intruso puede manipular la consulta SQL, obtener acceso no autorizado y visualizar información almacenada en la base de datos.

Posteriormente, modificaremos y corregiremos el código original aplicando las medidas adecuadas de seguridad con el objetivo de eliminar la vulnerabilidad. Finalmente, verificaremos que, tras la corrección, la web ya no puede ser explotada mediante SQL Injection.

Creación de la base de datos

1. Instalaremos Apache, PHP y MySQL en Ubuntu (Linux) mediante el comando **`sudo apt update && sudo apt install apache2 php libapache2-mod-php mariadb-server mariadbclient -y`**
2. El siguiente paso es crear la base de datos y la tabla de los usuarios. Los comandos a ejecutar son los siguientes:

1. `sudo systemctl start mysql` para iniciar MySQL.

2. `sudo mysql -u root -p` para acceder a MySQL.

3. Crear la base de datos y la tabla de usuarios:

```
CREATE DATABASE seguridad_db;
USE seguridad_db;
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  password VARCHAR(100) NOT NULL
);
```

4. Insertamos usuarios de prueba

1. `INSERT INTO users (username, password) VALUES ('admin', 'admin123'), ('usuario', 'pass123');`

```
(carles@kalicarles)-[~]
$ sudo mysql -u root -p

Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.8.3-MariaDB-1+b1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE seguridad_db;
Query OK, 1 row affected (0,001 sec)

MariaDB [(none)]> USE seguridad_db;
Database changed
MariaDB [seguridad_db]> CREATE TABLE users (
  → id INT AUTO_INCREMENT PRIMARY KEY,
  → username VARCHAR(50) NOT NULL,
  → password VARCHAR(100) NOT NULL
  → );
Query OK, 0 rows affected (0,238 sec)

MariaDB [seguridad_db]> INSERT INTO users (username, password) VALUES ('admin', 'admin123'), ('usuario', 'pass123');
Query OK, 2 rows affected (0,031 sec)
Records: 2 Duplicates: 0 Warnings: 0

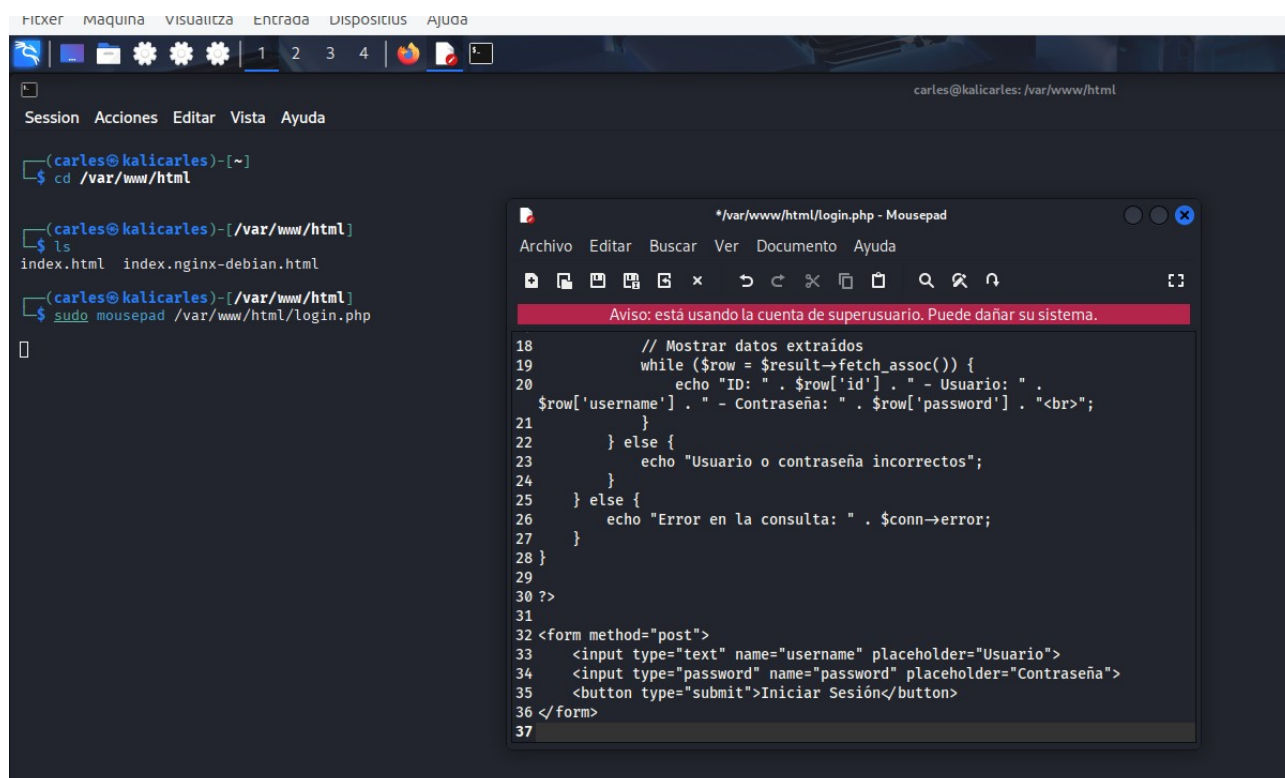
MariaDB [seguridad_db]> SET PASSWORD FOR 'root'@'localhost' = PASSWORD('root');
Query OK, 0 rows affected (0,031 sec)

MariaDB [seguridad_db]> █
```

Creación del archivo php en Apache

Con la base de datos ya creada faltará crear el archivo php que contendrá los errores los cuales serán explotados mediante la inyección SQL. Los pasos a seguir son:

1. `cd /var/www/html.`
2. `sudo mousepad /var/www/html/login.php.`
3. Copiamos el código vulnerable del ejercicio.



The screenshot shows a terminal window on the left and a text editor window on the right. The terminal window displays the following commands and output:

```
(carles@kalicarles)-[~]
$ cd /var/www/html

(carles@kalicarles)-[/var/www/html]
$ ls
index.html  index.nginx-debian.html

(carles@kalicarles)-[/var/www/html]
$ sudo mousepad /var/www/html/login.php
```

The text editor window, titled `* /var/www/html/login.php - Mousepad`, shows the following PHP code:

```
18 // Mostrar datos extraídos
19 while ($row = $result->fetch_assoc()) {
20     echo "ID: " . $row['id'] . " - Usuario: " .
    $row['username'] . " - Contraseña: " . $row['password'] . "<br>";
21 }
22 } else {
23     echo "Usuario o contraseña incorrectos";
24 }
25 } else {
26     echo "Error en la consulta: " . $conn->error;
27 }
28 }
29
30 ?>
31
32 <form method="post">
33     <input type="text" name="username" placeholder="Usuario">
34     <input type="password" name="password" placeholder="Contraseña">
35     <button type="submit">Iniciar Sesión</button>
36 </form>
37
```

4. Guardamos el archivo, establecemos los permisos adecuados y comprobamos el contenido del archivo login.php previamente reiniciando apache con `sudo systemctl restart apache2`. El código implementa un sistema de autenticación en PHP, conectándose a una base de datos MySQL para verificar.
5. credenciales enviadas a través de un formulario.

```
(carles@kalicarles)-[/var/www/html]
$ sudo chmod 755 /var/www/html/login.php

(carles@kalicarles)-[/var/www/html]
$ cat /var/www/html/login.php

<?php

$conn = new mysqli("localhost", "root", "root", "seguridad_db");

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = $_POST["username"];
    $password = $_POST["password"];

    $query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
    echo "Consulta ejecutada: " . $query . "<br>";

    $result = $conn->query($query);

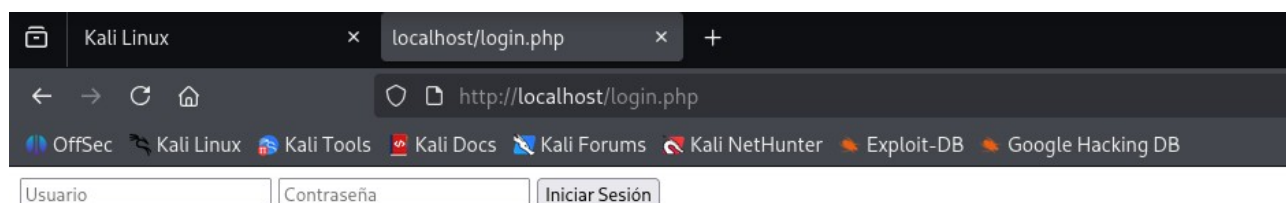
    if ($result) {
        if ($result->num_rows > 0) {
            echo "Inicio de sesión exitoso<br>";

            // Mostrar datos extraídos
            while ($row = $result->fetch_assoc()) {
                echo "ID: " . $row['id'] . " - Usuario: " . $row['username'] . " - Contraseña: " . $row['password'] . "<br>";
            }
        } else {
            echo "Usuario o contraseña incorrectos";
        }
    } else {
        echo "Error en la consulta: " . $conn->error;
    }
}

?>

<form method="post">
    <input type="text" name="username" placeholder="Usuario">
    <input type="password" name="password" placeholder="Contraseña">
    <button type="submit">Iniciar Sesión</button>
```

6. Desde el navegador web habrá que introducir la dirección de localhost junto a la página del login.php. `http://localhost/login.php`.

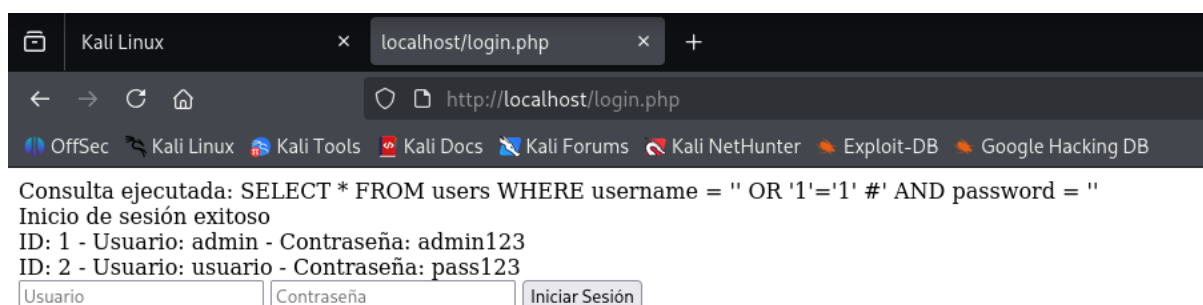


Exploitar SQL Injection

La primera explotación se realizará ingresando el siguiente código en el campo Usuario:

' OR '1'='1' #

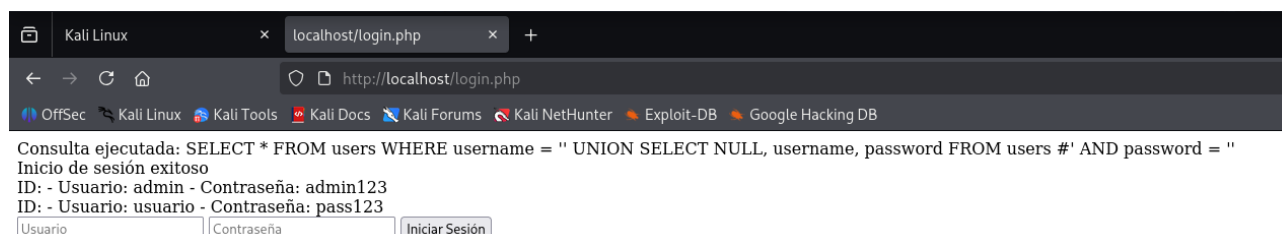
Sin necesidad de introducir una contraseña. Al hacerlo, la aplicación permitirá el acceso al sistema, demostrando que es posible iniciar sesión sin credenciales válidas.



La segunda explotación consistirá en utilizar el siguiente código en el campo Usuario:

```
' UNION SELECT NULL, username, password FROM users #
```

Esta inyección permite manipular la consulta original y obtener como resultado la visualización de todos los usuarios registrados en la base de datos junto con sus contraseñas.



¿Qué es lo que pasó técnicamente?

La consulta original es `SELECT * FROM users WHERE username = '$username' AND password = '$password'` y cuando inyectamos `' OR '1'='1' #`, se vuelve:

```
SELECT * FROM users WHERE username = " OR '1'='1' # ' AND password="
```

- `'1'='1'` siempre es verdadero
- El `#` comenta el resto
- Te permite entrar sin credenciales

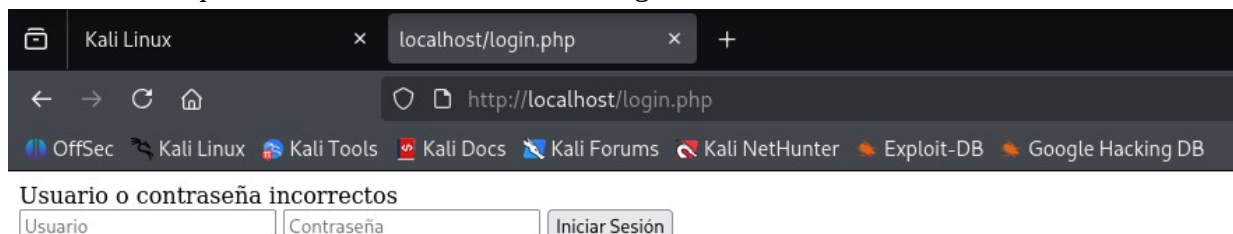
Cuando hacemos el UNION SELECT, agregamos otra consulta válida a la original para obtener más filas.

Solucionar las vulnerabilidades en el código php

Después de comprobar cómo es posible obtener acceso al sistema sin necesidad de una contraseña, procedemos a modificar el código del formulario de inicio de sesión para implementar las medidas necesarias que eviten este tipo de ataques de inyección SQL. Para ello los pasos a seguir son:

1. `sudo mousepad /var/www/html/login.php`.
2. Pegamos el nuevo código.
3. `sudo systemctl restart apache2`

4. Accedemos nuevamente a `http://localhost/login.php` y probamos el mismo bypass `'OR '1'='1' #`. En esta ocasión, el sistema ya no permite el acceso ni muestra un inicio de sesión exitoso. En su lugar, devuelve el mensaje “Usuario o contraseña incorrectos”, lo que confirma que la vulnerabilidad ha sido corregida.



Mejoras en el segundo código

Aunque es más seguro, aún hay cosas por mejorar si el sistema fuera real:

1. Las contraseñas siguen en texto plano.
 1. En un sistema real jamás debes mostrar contraseñas.
2. Muestra datos de los usuarios en pantalla
 1. No hay control de sesiones
 1. A pesar de corregir varios problemas de seguridad, no se establece una sesión segura (`session_start()`)
 2. después de una autenticación exitosa.

Código final totalmente seguro

```
<?php
session_start();
$conn = new mysqli("localhost", "root", "root", "seguridad_db");
if ($conn->connect_error) {
    die("Error de conexión: " . $conn->connect_error);
}
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // Sanitizar entrada
    $username = htmlspecialchars(trim($_POST["username"]), ENT_QUOTES, 'UTF-8');
    $password = trim($_POST["password"]);
    // Consulta segura para obtener usuario
    $stmt = $conn->prepare("SELECT id, username, password FROM users WHERE username
= ?");
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $result = $stmt->get_result();
    if ($result->num_rows === 1) {
        $row = $result->fetch_assoc();
        // Verificar contraseña usando password_verify
        if (password_verify($password, $row["password"])) {
            // Iniciar sesión segura
            $_SESSION["user_id"] = $row["id"];
```

```
$_SESSION["username"] = $row["username"];
echo "Inicio de sesión exitoso. Bienvenido, " . htmlspecialchars($row["username"]);
exit;
} else {
    echo "Usuario o contraseña incorrectos";
}
} else {
    echo "Usuario o contraseña incorrectos";
}
$stmt->close();
}
$conn->close();
?>
<form method="post">
    <input type="text" name="username" placeholder="Usuario" required>
    <input type="password" name="password" placeholder="Contraseña" required>
    <button type="submit">Iniciar Sesión</button>
</form>
```